PulseCart GitHub Actions Capstone

SRE / DevSecOps Engineer Instructions

A hands-on, certification-oriented, market-realistic lab for mastering GitHub Actions automation, CI/CD governance, supply-chain security, release evidence, and operational troubleshooting.

Field	Value
Role	SRE / DevSecOps engineer
Application development status	Assumed complete. Do not spend lab time building product features.
Local repository path	/Users/juan-rodriguez/Education/Github_Actions/gh-actions-lab-01/
Target product	PulseCart - B2B order, inventory, and fulfillment SaaS
Primary platform	GitHub Actions, GitHub Environments, GHCR, repository security controls
Generated on	2026-04-29

Core rule

Your job is to design, implement, harden, operate, and evaluate the automation platform. Codex or another development assistant owns the product application implementation. You may run, inspect, and test the app, but you should not turn this lab into a software development exercise.

Final target score: 90+ out of 100 with no automatic failure cap triggered.

Contents

Use this PDF as the project specification. Every completed phase should leave behind executable automation and written evidence.

1. Lab Boundaries and Handoff Contract

The development assistant should already have created a small but realistic PulseCart monorepo. Your responsibility begins at the operational boundary: repository governance, CI design, reusable automation, security controls, packaging, deployment, release evidence, and incident response.

1.1 What is assumed to exist before you start

Area	Expected handoff from development
Monorepo	apps/api, apps/web, packages/domain, infra/docker, docs, package manager workspace files, Dockerfiles, and local run instructions.
API	Health, order CRUD/read paths, fulfillment endpoint, metrics endpoint, integration-testable PostgreSQL and Redis behavior.
Web	Dashboard, orders view, deployments/status view, minimal UI tests or E2E path.
Shared package	Domain validation, order status transitions, pricing and inventory reservation rules with unit tests.
Scripts	Root scripts for lint, format:check, test:unit, test:integration, test:e2e, build, docker:build, smoke, and sbom.
Docs	Architecture, API contract, local development guide, testing guide, and developer handoff notes.
CI/CD status	No completed GitHub Actions workflows. The SRE/DevSecOps engineer owns the automation.

1.2 Your first validation only

You may validate the handoff locally. Do not rewrite the product unless a defect blocks the automation lab. When a defect blocks CI/CD, document it as an SRE finding and make the smallest possible fix.

```
cd /Users/juan-rodriguez/Education/Github_Actions/gh-actions-lab-01/
pnpm install
pnpm lint
pnpm format:check
pnpm test:unit
pnpm build
pnpm docker:build
pnpm smoke
```

If the repository uses npm or yarn instead of pnpm, adapt commands without changing the intent. The important point is that scripts are deterministic, non-interactive, and suitable for GitHub-hosted runners.

1.3 Out of scope for you

- Building product features that are not required for automation testing.
- Polishing UI design beyond what is needed for smoke and E2E tests.
- Changing the product architecture for preference rather than CI/CD reliability.
- Using long-lived cloud credentials merely because they are easier than OIDC.
- Accepting flaky workflows as normal. Flakiness is an operational defect.

2. SRE Mission and Architecture Objective

Build a professional GitHub Actions platform for PulseCart that could survive a real hiring conversation. Your work should demonstrate operational judgment, cost awareness, secure automation, and the ability to troubleshoot failures from evidence.

2.1 Target architecture

- Fast pull request CI that runs only relevant jobs and cancels stale runs.
- Deeper main-branch and nightly validation with integration, E2E, security, and risk reporting.
- Reusable workflows and custom actions to reduce duplication and demonstrate maintainability.
- Container build, smoke testing, SBOM generation, provenance/attestation where available, and digest-based deployment.
- Staging auto-deployment after successful package build.
- Production deployment through GitHub Environment approval and branch/tag restrictions.
- Rollback by immutable image digest, not mutable tags.
- Policy audit workflow that blocks unsafe workflow patterns.
- Troubleshooting drills that force diagnosis, root-cause documentation, and prevention rules.

2.2 Certification alignment

The lab is designed to exercise the skills measured by the GitHub Actions certification track: authoring workflows, consuming and troubleshooting workflows, authoring and maintaining actions, managing automation for organizations/enterprises, and securing/optimizing automation. Treat the GH-200 study guide as your map, but use implementation evidence as your proof.

2.3 Repository structure you should end with

```
.github/
  actions/
    setup-pulsecart/
    quality-gate/
    container-smoke-test/
  workflows/
    ci.yml
    reusable-node-checks.yml
    reusable-docker-build.yml
    pr-automation.yml
    package-image.yml
    deploy.yml
    release.yml
    nightly-maintenance.yml
    policy-audit.yml
    troubleshooting-drill.yml
docs/
  runbook.md
  security-review.md
  cost-review.md
  research-notes.md
  evidence/
    01-ci.md
    02-reusable-workflows.md
    03-custom-actions.md
    04-package-image.md
    05-deployment.md
    06-security-hardening.md
    07-troubleshooting.md
    08-cost-optimization.md
    09-certification-map.md
```

3. Non-Negotiable Operating Principles

These standards apply to every workflow unless a documented exception exists. A professional SRE writes automation that is secure by default, observable during failure, and inexpensive to run.

Principle	Requirement
Least privilege	Start workflows with permissions: contents: read or permissions: {}. Escalate per job only when needed, such as packages: write for GHCR or deployments: write for deployment records.
Third-party actions	No owner/repo@main or floating branches. Final hardening must pin non-local third-party actions to full-length commit SHAs.
Secrets	No plaintext secrets, no secrets in cache paths, no secrets printed in logs, no environment secrets available before environment approval.
PR security	Do not place untrusted PR metadata directly inside shell scripts. Pass values through environment variables or custom JavaScript actions.
OIDC preference	Use OIDC for real cloud credentials where feasible. Avoid long-lived cloud access keys.
Cost control	Every workflow needs timeout-minutes, concurrency, path filters where useful, short retention-days, and max-parallel tuning where useful.
Observability	Every major workflow writes GITHUB_STEP_SUMMARY content with status, decisions, artifacts, digests, links, and next steps.
Reproducibility	Use lockfiles, deterministic scripts, immutable image digests, and documented runner assumptions.
Evidence	Every phase ends with a docs/evidence entry containing run links, results, failures, fixes, and final conclusion.

3.1 Hard security checks

- No permissions: write-all in final workflows.
- No pull_request_target unless you can explain the risk and prove the workflow never checks out or executes untrusted code with a privileged token.
- No cache path may include .env, credentials, secrets, tokens, private keys, or cloud config files.
- Every deployment job must reference a GitHub environment.
- Production must have required reviewers or equivalent protection.
- Rollback must deploy a chosen digest and record why the rollback occurred.
- Every workflow that uploads artifacts must set retention-days.
- Every shell step that handles external input must quote safely and avoid expression injection.

4. Repository and Environment Setup

4.1 Repository governance tasks

- Create or confirm the default branch is main.
- Configure branch protection or rulesets for main: require PRs, require status checks after CI exists, block force pushes, and prefer signed commits if practical.
- Create labels for area:api, area:web, area:domain, area:infra, risk:high, ci:needs-evidence, release, deployment, and security.
- Create issue templates for incident, nightly failure, and release readiness if useful.
- Create a README section explaining that application development is separate from this SRE/DevSecOps lab.

4.2 GitHub Environments

Environment	Required configuration
staging	Deployment branch policy allows main. No manual approval required. Environment URL points to the simulated dashboard or real staging target.
production	Requires approval. Restrict deployment to protected branches and/or semantic version tags. Store production-only secrets here if using real cloud.

4.3 Variables, secrets, and credentials

- Prefer repository variables for non-sensitive values such as API base URL, environment names, and simulated deployment paths.
- Use repository or environment secrets only for sensitive values that cannot be avoided.
- For GHCR publishing, prefer GITHUB_TOKEN with packages: write rather than a PAT.
- For cloud deployments, configure a federated identity trust and use id-token: write only in the job that needs OIDC.
- Record every secret/variable purpose in docs/security-review.md without exposing secret values.

5. Implementation Phases and Workflow Requirements

Complete the phases in order. Later phases may refactor earlier work, but every phase must leave behind working automation and evidence.

Phase 0 - Handoff Validation

Confirm that the application handoff is ready for automation. This is not a feature-development phase.

Triggers / entry point:

- Manual local execution before CI is built.

Deliverables:

- docs/evidence/00-handoff-validation.md or a section inside docs/evidence/01-ci.md.
- A short list of scripts tested locally and their results.
- A short list of blocking defects, if any, with the smallest fixes applied.

Strict requirements:

- Run install, lint, format:check, unit tests, build, docker build, and smoke locally.
- Do not create GitHub Actions yet unless local scripts pass or defects are documented.
- Document package manager, Node version, Docker version if relevant, and any required environment variables.

Research tasks:

- What makes a script CI-friendly?
- Which scripts require service containers?
- Which failures are app defects versus automation defects?

Acceptance test:

- A fresh clone can run the documented local commands.
- Any required database/Redis dependency is available through docker compose or documented env vars.

Evidence to capture:

- Command output snippets, local screenshots if helpful, and final handoff conclusion.

Phase 1 - ci.yml: Pull Request and Push CI

Create fast, reliable validation for pull requests and main branch pushes.

Triggers / entry point:

- pull_request
- push to main
- workflow_dispatch

Deliverables:

- .github/workflows/ci.yml
- Coverage artifacts, test reports, build artifacts, and job summaries.
- docs/evidence/01-ci.md

Strict requirements:

- Use path filters so API-only, web-only, and shared-domain changes run the right jobs.
- Use concurrency to cancel stale CI runs for the same PR or branch.
- Use a small matrix on PRs and deeper validation on main if needed.
- Use PostgreSQL and Redis service containers for integration tests.
- Cache dependencies using a lockfile-based key.
- Upload coverage/test/build artifacts with retention-days.
- Use timeout-minutes on every job.
- Expose useful outputs such as changed areas, coverage percent, and build version.
- Write a Markdown summary table with status, coverage, changed areas, artifacts, and skipped jobs.

Research tasks:

- pull_request versus pull_request_target.
- Service container health checks.
- Matrix include/exclude, fail-fast, max-parallel.
- GITHUB_OUTPUT for passing step/job data.
- Safest way to handle PR title or branch names in shell.

Acceptance test:

- Create one PR touching apps/web only, one touching apps/api only, and one touching packages/domain. Correct jobs must run or skip.
- Break one unit test and confirm the failure is easy to diagnose from summary plus artifacts.

Evidence to capture:

- Workflow run URLs, summary screenshot or copied Markdown, artifact names, coverage values, and path-filter proof.

Phase 2 - reusable-node-checks.yml

Extract repeatable Node validation into a reusable workflow.

Triggers / entry point:

- workflow_call

Deliverables:

- .github/workflows/reusable-node-checks.yml
- At least two caller workflows using it.
- docs/evidence/02-reusable-workflows.md

Strict requirements:

- Define typed inputs: node-version, workspace, run-integration, coverage-threshold, upload-artifacts.
- Define outputs: coverage-percent, test-report-path, build-version.
- Do not require secrets by default.
- Map step output to job output to workflow output correctly.
- Call from ci.yml and nightly-maintenance.yml.

- Document when reusable workflows are better than composite actions.

Research tasks:

- workflow_call input types and defaults.
- Reusable workflow output mapping.
- How secrets behave in nested reusable workflows.
- Why environment secrets cannot be passed through workflow_call like normal secrets.

Acceptance test:

- Both caller workflows consume outputs and report them in job summaries.
- A failed threshold in the reusable workflow fails the caller workflow clearly.

Evidence to capture:

- Caller YAML snippets, run links, output values, and one failure example.

Phase 3 - Custom Composite Action: setup-pulsecart

Remove repeated setup steps and make workflow files cleaner.

Triggers / entry point:

- Used as a local action step from workflows.

Deliverables:

- .github/actions/setup-pulsecart/action.yml
- Usage in at least three workflows.
- docs/evidence/03-custom-actions.md

Strict requirements:

- Caller performs checkout before invoking this action.
- Action installs runtime or validates runtime, enables package manager, restores dependency cache, installs dependencies, and prints dependency summary.
- Inputs should include node-version, package-manager, cache-key-prefix, and install-command if useful.
- No secrets accepted.
- Use clear output variables if needed.

Research tasks:

- Composite action metadata syntax.
- Composite action limitations compared with reusable workflows.
- When local actions are appropriate.

Acceptance test:

- ci.yml, package-image.yml, and nightly-maintenance.yml use the composite action successfully.

Evidence to capture:

- Before/after duplication comparison and three run links.

Phase 4 - pr-automation.yml: Secure PR Hygiene

Automate PR labels, quality comments, and description checks without executing attacker-controlled input.

Triggers / entry point:

- pull_request
- issues
- issue_comment
- workflow_dispatch

Deliverables:

- .github/workflows/pr-automation.yml
- PR title and body validation.
- Area labels and quality report comments.
- docs/evidence/06-security-hardening.md section.

Strict requirements:

- Validate PR title format.
- Label PRs based on changed files.
- Fail if PR body lacks problem statement, test evidence, or rollback notes.
- Use least-privilege permissions: contents: read plus pull-requests/issues write only where needed.
- Avoid pull_request_target unless the risk model is fully documented and mitigated.
- Use safe handling for PR metadata.

Research tasks:

- Script injection through malicious PR metadata.
- Permissions needed to write PR comments and labels.
- Inline shell versus JavaScript action for untrusted strings.
- GITHUB_TOKEN versus PAT.

Acceptance test:

- Create a PR titled: feat: test"; echo "pwned"; #. The workflow must not execute injected shell content.
- Open a PR with missing rollback notes and confirm the workflow fails with a useful message.

Evidence to capture:

- Malicious-title run link, logs proving no injection, permission block excerpt, and final conclusion.

Phase 5 - Custom JavaScript Action: quality-gate

Build a real JavaScript action that performs CI quality decisions and emits annotations.

Triggers / entry point:

- Called from ci.yml, reusable-node-checks.yml, or package/release workflows.

Deliverables:

- .github/actions/quality-gate/action.yml

- Source and bundled dist if the action requires it.
- Action README.
- docs/evidence/03-custom-actions.md section.

Strict requirements:

- Inputs: coverage-percent, coverage-threshold, test-report-path, changed-files-json, mode.
- Outputs: passed, reason, risk-level.
- Fail when coverage is below threshold.
- Warn when many files changed without E2E evidence.
- Generate annotations for failed test files.
- Write structured Markdown summary.
- Use @actions/core and @actions/github.

Research tasks:

- How JavaScript actions set outputs.
- How annotations work.
- Why bundled dist files are normally committed.
- How pre/post hooks work.

Acceptance test:

- Lower coverage below threshold and confirm the action fails with a useful annotation and output.
- Restore coverage and confirm pass behavior.

Evidence to capture:

- Action logs, annotation screenshot or copied annotation, output values, and README excerpt.

Phase 6 - Custom Docker Action: container-smoke-test

Validate built container images consistently using a Docker action.

Triggers / entry point:

- Called from package-image.yml after image build.

Deliverables:

- .github/actions/container-smoke-test/action.yml
- Dockerfile and entrypoint.
- Smoke output JSON artifact.
- docs/evidence/03-custom-actions.md section.

Strict requirements:

- Input image reference or tar path.
- Run container, wait for /health, run smoke checks, capture logs, write output JSON, fail clearly.
- Works on Linux runners only.
- No secret input required.

- Output health status, endpoint results, container logs path, and elapsed time.

Research tasks:

- When Docker actions are better than JavaScript actions.
- Why Docker actions are Linux-runner-only.
- How Docker action inputs become args or environment variables.

Acceptance test:

- Use the Docker action in package-image.yml against the API image and verify health plus orders smoke checks.

Evidence to capture:

- Smoke JSON, logs artifact, failed-container example if practical, and run URL.

Phase 7 - package-image.yml: Build, Scan, Attest, Publish

Create a supply-chain-aware image packaging workflow.

Triggers / entry point:

- push to main
- push tags v*
- workflow_dispatch
- workflow_call

Deliverables:

- .github/workflows/package-image.yml
- Optional reusable-docker-build.yml
- GHCR images or documented simulation.
- SBOM and attestation evidence where available.
- docs/evidence/04-package-image.md

Strict requirements:

- Build API and web images.
- Tag with commit SHA, branch name, semantic version on tags, and latest only from default branch.
- Push to GHCR when repository permissions allow.
- Export immutable image digest as job/workflow output.
- Run container-smoke-test.
- Generate SBOM.
- Generate artifact attestation or document why it is unavailable.
- Upload build metadata artifact with retention-days.
- Use packages: write only in publishing jobs, and id-token/attestations permissions only where needed.

Research tasks:

- Image tag versus image digest.
- Why digest-based deployment is safer than tag-based deployment.

- GHCR permissions with GITHUB_TOKEN.
- Artifact attestation permissions.
- How to verify an attestation with GitHub CLI.

Acceptance test:

- After a successful run, identify the exact digest intended for staging.
- Tamper with smoke health and confirm package workflow fails before deployment.

Evidence to capture:

- Image names, tags, digest, SBOM artifact, attestation verification output if available, and summary.

Phase 8 - deploy.yml: Staging, Production, Rollback

Build a governed deployment pipeline using GitHub Environments.

Triggers / entry point:

- workflow_run from package-image.yml
- workflow_dispatch

Deliverables:

- .github/workflows/deploy.yml
- docs/deployments/staging.json
- docs/deployments/production.json
- Deployment manifests as artifacts.
- docs/evidence/05-deployment.md

Strict requirements:

- Auto-deploy staging from main after successful package-image workflow.
- Production deploy is manual or tag-based and requires environment approval.
- Rollback input image_digest deploys a previous digest.
- One deployment per environment at a time via concurrency.
- Deployment job references the correct environment.
- Set or summarize deployment URL.
- Store deployment manifest artifact and update simulated deployment JSON.
- Use digest-based deployment, not latest.

Research tasks:

- Environment secrets versus repository secrets.
- Branch restrictions for production.
- workflow_run versus workflow_dispatch.
- OIDC and short-lived credentials.
- Safe rollback by digest.

Acceptance test:

- Deploy staging automatically, deploy production manually after approval, then roll back production to an older digest.
- Confirm a second production run queues or cancels according to the concurrency strategy.

Evidence to capture:

- Approval screenshot/copy, manifest artifact, deployment JSON, digest before/after rollback, run links.

Phase 9 - release.yml: Release Management

Create an idempotent release workflow with artifacts and release evidence.

Triggers / entry point:

- push tags v*
- workflow_dispatch

Deliverables:

- .github/workflows/release.yml
- GitHub Release with attachments.
- docs/evidence/release section or docs/evidence/04-package-image.md

Strict requirements:

- Validate semantic version.
- Run or require final quality gate.
- Download build artifacts from previous workflow or rebuild deterministically.
- Create GitHub Release idempotently.
- Attach deployment manifest, SBOM, test report, coverage report, and changelog.
- Write release summary to GITHUB_STEP_SUMMARY.
- Use contents: write only for release creation job.

Research tasks:

- GitHub Release versus package versus artifact versus deployment.
- Release permissions versus package permissions.
- Avoiding duplicate releases.

Acceptance test:

- Create v0.1.0, then intentionally attempt v0.1.0 again. Workflow must skip safely or fail clearly without corrupting release assets.

Evidence to capture:

- Release URL, attached asset list, duplicate-run behavior, summary.

Phase 10 - nightly-maintenance.yml

Run slower validation and risk reporting without slowing everyday PR work.

Triggers / entry point:

- schedule

- workflow_dispatch

Deliverables:

- .github/workflows/nightly-maintenance.yml
- Risk report artifact.
- Issue creation/update behavior.
- docs/evidence/nightly section.

Strict requirements:

- Run full matrix including one experimental version.
- Run dependency audit.
- Run E2E tests.
- Check stale artifacts and caches where possible.
- Produce risk report.
- Open or update exactly one issue when nightly health fails.
- Use max-parallel to control cost.
- Use UTC-aware schedule and document local equivalent.

Research tasks:

- schedule uses UTC.
- Scheduled workflows and inactivity.
- Updating one existing issue instead of creating duplicates.
- max-parallel for cost control.

Acceptance test:

- Force a nightly failure and confirm exactly one issue is opened or updated.
- Restore health and confirm the summary communicates recovery.

Evidence to capture:

- Risk report, issue link, run links, matrix details, cost/time notes.

Phase 11 - policy-audit.yml

Enforce professional workflow hygiene automatically.

Triggers / entry point:

- pull_request touching .github/**
- workflow_dispatch
- weekly schedule

Deliverables:

- .github/workflows/policy-audit.yml
- Policy script or checks.
- docs/evidence/06-security-hardening.md

Strict requirements:

- Fail on owner/repo@main or floating third-party action refs.
- Fail on permissions: write-all.
- Fail on missing timeout-minutes unless exception is documented.
- Fail on deployment jobs without environment.
- Fail on cache paths containing .env, secrets, credentials, tokens, or keys.
- Fail on direct shell use of github.event.pull_request.title.
- Fail on missing concurrency unless exception is documented.
- Fail on upload-artifact steps without retention-days.

Research tasks:

- Workflow static analysis strategies.
- Known GitHub Actions security anti-patterns.
- How to avoid false positives while keeping policy strict.

Acceptance test:

- Create a branch with permissions: write-all and uses: some/action@main. The audit must fail.
- Fix the branch and confirm the audit passes.

Evidence to capture:

- Bad workflow sample, failed audit output, fixed output, prevention rationale.

Phase 12 - troubleshooting-drill.yml

Practice exam-style diagnostics and operational root-cause analysis.

Triggers / entry point:

- workflow_dispatch

Deliverables:

- .github/workflows/troubleshooting-drill.yml
- At least four completed drills.
- docs/evidence/07-troubleshooting.md

Strict requirements:

- Support modes: broken-cache, broken-service-container, broken-output, broken-permission, broken-expression, broken-artifact.
- Each mode triggers a controlled failure.
- You diagnose from logs and summaries.
- You document root cause, fix, and prevention.
- You add at least one permanent prevention rule to policy-audit.yml after a drill.

Research tasks:

- Debug logging.
- Rerunning one failed matrix job.

- Inspecting contexts safely.
- Parse-time expression errors versus runtime shell errors.

Acceptance test:

- Complete at least four incident drills and document root cause, fix, prevention, and evidence link.
- For at least one drill, rerun only the failed job where appropriate.

Evidence to capture:

- Incident table, logs excerpt, fix commit, prevention rule, final lesson.

6. Mandatory Research Backlog

Create docs/research-notes.md and record a short entry for every item below. Use official documentation first. You are expected to research during the lab instead of pretending everything is already known.

6.1 Research note template

```
Topic:
Problem I had:
Official source:
What I learned:
Decision I made:
How I validated it:
```

6.2 Required topics

1. workflow_call inputs, secrets, and outputs
2. workflow_dispatch input types and defaults
3. matrix include/exclude/fail-fast/max-parallel
4. service containers with PostgreSQL health checks
5. GITHUB_ENV versus GITHUB_OUTPUT versus GITHUB_STEP_SUMMARY
6. artifacts versus caches
7. GITHUB_TOKEN permissions versus PAT
8. OIDC id-token permissions and subject restrictions
9. artifact attestations and SBOMs
10. composite versus JavaScript versus Docker actions
11. pull_request versus pull_request_target
12. workflow_run versus repository_dispatch versus workflow_dispatch
13. environment protection rules
14. action SHA pinning
15. script injection prevention
16. self-hosted runner risks and labels
17. starter workflows versus reusable workflows versus composite actions
18. managing artifacts/logs/caches through API or GitHub CLI

7. Evidence Pack Requirements

Evidence is not optional. A professional automation project should be auditable after the fact. Each file should read like a compact operational report, not like a diary.

7.1 Required evidence files

```
docs/evidence/01-ci.md
docs/evidence/02-reusable-workflows.md
docs/evidence/03-custom-actions.md
docs/evidence/04-package-image.md
docs/evidence/05-deployment.md
docs/evidence/06-security-hardening.md
docs/evidence/07-troubleshooting.md
docs/evidence/08-cost-optimization.md
docs/evidence/09-certification-map.md
```

7.2 Required content in each evidence file

- Workflow run links and relevant commit SHAs.
- Screenshots or copied GITHUB_STEP_SUMMARY content.
- What passed and what failed.
- What you changed after failure.
- What you researched and what source guided the decision.
- Artifacts produced, retention period, and why those artifacts matter.
- Final conclusion with risk level: low, medium, or high.

7.3 docs/evidence/09-certification-map.md

Map your implemented evidence to GH-200 topics. For each topic, include the exact workflow, action, incident, or document that proves hands-on experience.

Certification skill area	Evidence to map
Workflow authoring	ci.yml, matrix, service containers, expressions, contexts, outputs, summaries.
Workflow consumption/troubleshooting	troubleshooting-drill.yml, failed run diagnosis, rerun evidence, logs.
Actions authoring	Composite setup action, JavaScript quality-gate, Docker smoke-test action.
Enterprise/platform management	Branch protection/rulesets, environments, policy audit, runner/cost decisions.
Security and optimization	Least permissions, OIDC, attestation/SBOM, action pinning, cache hygiene, PR injection prevention.

8. Strict Evaluation Matrix

Apply automatic failure caps first. Then calculate the 100-point score. A cap means the final score cannot exceed the cap even if the raw score is higher.

8.1 Automatic failure caps

Failure	Maximum score
Main CI does not pass	60
No reusable workflow	70
No custom action	70
No deployment environment	75
Secrets printed in logs	50
Long-lived cloud credentials used where OIDC was feasible	75
Production deploy has no approval/protection	75
Workflows use write-all without strong justification	80
Any third-party @main reference remains in final version	80
No evidence pack	70
No research notes	75
Project cannot be cloned and run by another developer	65

8.2 Scoring rubric - 100 points

Category	Points	Excellent standard
Product/repo foundation validation	8	You verify the app handoff, local scripts, Docker Compose, README, docs, and minimal product behavior without turning this into feature development.
CI correctness	14	PR and push CI are deterministic, path-aware, fast, artifact-producing, summary-rich, and fail for meaningful reasons.
Matrix and service containers	8	Matrix is useful but cost-aware; PostgreSQL/Redis service containers are healthy and reliable.
Reusable workflows	10	workflow_call inputs, outputs, and callers are implemented cleanly with deliberate secret handling.
Custom actions	14	Composite, JavaScript, and Docker actions are implemented, documented, tested, versionable, and used in real workflows.
Packaging and release	10	Images are built, tagged, smoke-tested, digests captured, releases created idempotently, SBOM/attestation metadata produced where available.
Deployment pipeline	10	Staging auto-deploy works, production approval works, rollback by digest works, manifests and deployment records are clear.
Security hardening	14	Minimal permissions, safe PR handling, no secret leakage, action pinning, OIDC design, safe cache paths, policy audit.
Troubleshooting and observability	8	Summaries, incident drills, failed matrix diagnosis, runbooks, logs, and artifacts are easy to navigate.
Cost/time optimization	4	Concurrency, path filters, reduced PR matrix, nightly full matrix, retention, cache keys, timeouts, and max-parallel are intentional.
Research quality	6	Research notes are official-docs-based, decision-oriented, and connected to implementation choices.

8.3 Grade interpretation

Score	Meaning
95-100	Market-ready. Portfolio-quality SRE/DevSecOps automation.
90-94	Certification-ready. Minor polish remains.
80-89	Good learning project but not yet senior-quality.
70-79	Major gaps. Redo weak areas before attempting the exam.
Below 70	Lab incomplete. Do not treat as certification preparation yet.

9. Final Definition of Done

[ ] A fresh clone can install, test, build, and run locally.

[ ] Pull request CI runs fast and skips irrelevant work.

[ ] Main branch CI runs deeper validation.

[ ] Reusable workflow is called from at least two workflows.

[ ] Composite action is used in at least three workflows.

[ ] JavaScript quality-gate action fails and passes correctly.

[ ] Docker smoke-test action validates a container image.

[ ] Images are published to GHCR or package publishing is simulated with clear evidence.

[ ] Staging deployment runs automatically after successful packaging.

[ ] Production deployment requires environment approval.

[ ] Rollback by image digest works.

[ ] Policy audit catches unsafe workflow patterns.

[ ] At least four troubleshooting drills are completed.

[ ] Every important workflow writes a useful job summary.

[ ] Evidence pack and research notes are complete.

[ ] Final score is 90+ using the matrix above.

9.1 Recommended execution order

1. Validate app handoff locally.
2. Create ci.yml with path filters, services, cache, artifacts, summaries.
3. Extract setup logic into setup-pulsecart composite action.
4. Build reusable-node-checks.yml and call it from CI and nightly.
5. Build quality-gate JavaScript action.
6. Add secure PR automation and script-injection proof.
7. Build Docker smoke-test action and package-image.yml.
8. Add SBOM, image digests, attestation where available, and build metadata.
9. Add deploy.yml with staging, production, and rollback.
10. Add release.yml.
11. Add nightly-maintenance.yml.
12. Add policy-audit.yml.
13. Run troubleshooting drills.
14. Harden everything: permissions, SHA pinning, retention, timeouts, concurrency, summaries.
15. Score yourself and redo any weak category.

10. Official Reference List

Use official sources first when completing research notes. These references were used to shape the lab requirements and should be verified again if GitHub changes product behavior.

Topic	URL
GH-200 study guide	https://learn.microsoft.com/en-us/credentials/certifications/resources/study-guides/gh-200
Reusable workflows	https://docs.github.com/en/actions/how-tos/reuse-automations/reuse-workflows
Workflow syntax	https://docs.github.com/actions/using-workflows/workflow-syntax-for-github-actions
Secure use reference / action pinning	https://docs.github.com/en/actions/reference/security/secure-use
Deployment environments	https://docs.github.com/en/actions/reference/workflows-and-actions/deployments-and-environments
Managing environments	https://docs.github.com/actions/deployment/targeting-different-environments/using-environments-for-deployment
OIDC in cloud providers	https://docs.github.com/actions/deployment/security-hardening-your-deployments/configuring-openid-connect-in-cloud-providers
Artifact attestations	https://docs.github.com/actions/security-for-github-actions/using-artifact-attestations/using-artifact-attestations-to-establish-provenance-for-builds
Workflow commands	https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-commands
Dependency caching	https://docs.github.com/en/actions/reference/workflows-and-actions/dependency-caching
Workflow artifacts	https://docs.github.com/en/actions/tutorials/store-and-share-data
Composite actions	https://docs.github.com/actions/creating-actions/creating-a-composite-action

End of specification.